

Chapitre 4

Processus & Threads

Programmation (2/2)

Système d'Exploitation et Programmation Système

Pr. MANALE Boughanja

m.boughanja@ump.ac.ma

Table des matières

1	Création d'un Processus et Threads	2
1.1	Rappels Fondamentaux	2
1.2	Comparaison Processus vs Thread	2
2	Famille de Fonctions exec()	2
2.1	Exécution d'un Code Différent	2
2.2	Les Variantes de exec()	3
3	Ordre d'Exécution des Processus	4
4	Synchronisation des Processus	5
4.1	Attente des Processus Fils	5
4.1.1	La fonction wait()	5
4.1.2	La fonction waitpid()	6
4.2	Valeurs du Paramètre pid dans waitpid()	6
4.3	Problèmes Sans Synchronisation	6
4.3.1	Cas 1 : Processus Orphelin	6
4.3.2	Cas 2 : Processus Zombie	6
4.4	Remarques Importantes sur la Synchronisation	7
5	Retour du Statut des Processus Fils	8
5.1	Mécanisme de Retour	8
5.2	Traitement de la Valeur Retour status	8
5.3	Exemple Complet de Synchronisation	8
6	Résumé et Bonnes Pratiques	9
7	Tableau Récapitulatif des Fonctions	11

1 Création d'un Processus et Threads

1.1 Rappels Fondamentaux

Rappel

Processus : Programme en cours d'exécution, disposant de sa propre mémoire et de ses ressources.

Thread : Sous-ensemble d'un processus, partage la mémoire du processus parent.

1.2 Comparaison Processus vs Thread

Critère	Processus	Thread
Mémoire	Indépendante	Partagée
Communication	IPC (Inter-Process Communication)	Accès direct aux variables partagées
Création (C)	<code>fork()</code>	<code>pthread_create()</code>
Création (Python)	<code>os.fork()</code>	<code>threading.Thread()</code>
Valeur retour	-1 : erreur 0 : processus fils >0 : processus père (PID du fils)	Via <code>pthread_exit</code> / <code>pthread_join</code>
Type valeur retour	<code>int</code>	<code>void *</code>

Note Importante

Points importants :

- Sous Unix, le processus `init` est le seul qui n'a pas de parent
- Sous Windows, la création des processus est effectuée par l'appel à la méthode `CreateProcess()`

2 Famille de Fonctions exec()

2.1 Exécution d'un Code Différent

Définition

L'appel à `fork()` provoque la duplication du code du processus père pour la création du processus fils.

Pour créer un processus fils disposant d'un **code différent** de celui du processus père, on utilise la famille de fonctions `exec()`.

Note Importante

Analogie : Par défaut, l'enfant hérite du code et des ressources du parent, comme un enfant hérite des traits génétiques de ses parents. Si on souhaite que l'enfant ait un comportement différent (exécute un code différent), il faut lui attribuer un rôle ou un code spécifique après sa création, via `exec()`.

2.2 Les Variantes de `exec()`

lightgreengreen!40 Fonction	Description
<code>lightgreenint execl(const char *path, const char *arg, ...)</code>	Lance un programme en indiquant son chemin complet et une liste d'arguments séparés .
<code>lightgreenint execlp(const char *file, const char *arg, ...)</code>	Lance un programme en cherchant son exécutable dans le PATH et en passant une liste d'arguments séparés .
<code>lightgreenint execl(const char *path, const char *arg, ..., char *const envp[])</code>	Lance un programme avec une liste d'arguments séparés tout en permettant de définir un environnement personnalisé (<code>envp</code>).
<code>lightgreenint execv(const char *path, char *const argv[])</code>	Lance un programme en indiquant son chemin complet et un tableau d'arguments (<code>argv</code>).
<code>lightgreenint execvp(const char *file, char *const argv[])</code>	Lance un programme en cherchant l'exécutable dans le PATH et en passant un tableau d'arguments (<code>argv</code>).

Exemple

Exemple d'utilisation de `execl` :

```

1 #include <unistd.h>
2 #include <stdio.h>
3
4 int main() {
5     pid_t pid = fork();
6
7     if (pid == 0) {
8         // Processus fils

```

```
9     execl("/bin/ls", "ls", "-l", NULL);
10    // Si exec réussit, le code ci-dessous n'est jamais
11    // execute
12    perror("execl a echoue");
13 } else if (pid > 0) {
14    // Processus pere
15    printf("Je suis le pere, PID fils: %d\n", pid);
16 } else {
17    perror("fork a echoue");
18 }
19
20 return 0;
21 }
```

3 Ordre d'Exécution des Processus

Attention

Points critiques concernant l'ordre d'exécution :

- Les codes du processus père et des processus fils **ne sont plus exécutés dans l'ordre d'écriture**
- Un même code peut être exécuté par **plusieurs processus**
- L'exécution des processus est **entrelacée par le scheduler**
- L'exécution d'un même programme peut donc provoquer des **affichages/résultats différents**

Conséquence importante : L'ordonnanceur décide de l'ordre d'exécution des processus, donc le résultat peut changer à chaque exécution.

Exemple

Exemple illustrant l'ordre d'exécution non déterministe :

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main() {
5     printf("Debut\n");
6
7     fork();
8
9     printf("Message\n");
10
11     return 0;
12 }
```

Exécutions possibles :

— Possibilité 1 :

Debut
Message (pere)
Message (fils)

— Possibilité 2 :

Debut
Message (fils)
Message (pere)

4 Synchronisation des Processus

4.1 Attente des Processus Fils

Définition

Trois fonctions sont à disposition pour surveiller le changement d'état de processus fils (terminaison, pause, continuation) :

1. `pid_t wait(int* status)`
2. `pid_t waitpid(pid_t pid, int *status, int options)`
3. D'autres variantes pour des cas spécifiques

4.1.1 La fonction `wait()`

Note Importante

`pid_t wait(int* status)`

Description : C'est la forme la plus simple. Le processus père s'arrête (il est bloqué) et attend qu'un de ses processus fils se termine.

Comportement :

- Si plusieurs fils se terminent, le système en choisit un **arbitrairement**
- Pour attendre N processus fils, il faut **N appels** à `wait()`

Rôle : Ces fonctions permettent au père non seulement de synchroniser sa propre terminaison, mais aussi de récupérer le statut de sortie du fils (le code passé à `exit()`).

4.1.2 La fonction `waitpid()`

Note Importante

`pid_t waitpid(pid_t pid, int *status, int options)`

Description : Cette fonction est plus flexible. Le paramètre `pid` permet de spécifier quel processus fils attendre.

C'est indispensable lorsque l'ordre de terminaison des fils est critique pour la logique de l'application.

4.2 Valeurs du Paramètre `pid` dans `waitpid()`

Valeur <code>pid</code>	Comportement
<code>-1</code>	Attente d'un processus fils dont le <code>groupid</code> est <code>-pid</code> (voir <code>setpgid()</code>).
<code>-1</code>	Attente d'un processus fils (semblable à <code>wait()</code>).
<code>0</code>	Attente d'un processus fils du même groupe que le processus père.
<code>> 0</code>	Attente du processus fils ayant le PID spécifié.

4.3 Problèmes Sans Synchronisation

Attention

Sans appel à `wait()`, deux cas possibles si aucune synchronisation entre le processus père et les processus fils :

1. Le processus père se termine avant ses processus fils → **Processus Orphelins**
2. Le(s) processus fils se termine(nt) avant le processus père → **Processus Zombies**

4.3.1 Cas 1 : Processus Orphelin

Définition

Processus Orphelin : Un processus dont le parent s'est terminé avant lui.

Solution du système : Le processus orphelin est automatiquement adopté par le processus `init` (`PID = 1`) qui devient son nouveau parent.

4.3.2 Cas 2 : Processus Zombie

Définition

Processus Zombie : Un processus qui s'est terminé mais dont le statut de sortie n'a pas été récupéré par son parent.

Comportement :

- Si un processus fils n'est pas attendu, il reste zombie :

- Jusqu'à la fin du processus père (si aucun `wait()` du père)
- Ou jusqu'à un appel à `wait()` du père récupérant la fin du processus fils
- La commande `ps aux` : processus zombies identifiés par Z+

Exemple

Exemple créant un processus zombie :

```

1 #include <stdio.h>
2 #include <unistd.h>
3 #include <stdlib.h>
4
5 int main() {
6     pid_t pid = fork();
7
8     if (pid == 0) {
9         // Processus fils
10        printf("Fils: Je me termine\n");
11        exit(0);
12    } else if (pid > 0) {
13        // Processus pere qui ne fait pas wait()
14        printf("Pere: Je ne fais pas wait()\n");
15        sleep(30); // Le fils devient zombie pendant 30
16                   secondes
17    }
18
19    return 0;
20 }
```

Vérification : Pendant l'exécution, utilisez `ps aux | grep Z` pour voir le processus zombie.

4.4 Remarques Importantes sur la Synchronisation

Attention

Règles essentielles :

- L'appel à `sleep()` **ne doit jamais être utilisé** pour synchroniser des processus
- La synchronisation doit se faire à l'aide de **fonctions de synchronisation adéquates** comme `wait()`

Note Importante

Comportement de `wait()` avec plusieurs fils :

- Un appel à `wait()` récupère la terminaison d'un des processus fils
- Si plusieurs processus fils sont terminés, le système en choisit un **arbitrairement**

- Dans de nombreux cas, l'ordre de terminaison importe : vous devez utiliser `waitpid()` pour spécifier le processus à attendre

5 Retour du Statut des Processus Fils

5.1 Mécanisme de Retour

Définition

Les fonctions `wait()` et `waitpid()` permettent au processus père de récupérer le statut d'un processus fils.

Côté processus fils :

```
1 void exit(int status)
```

Côté processus père :

```
1 int status;
2 wait(&status);
```

5.2 Traitement de la Valeur Retour status

Note Importante

La valeur `status` peut être traitée par les fonctions suivantes :

lightbluemaincolor !30 Ma- cro	Description
lightblueWIFEXITED(status)	Retourne vrai si le processus fils s'est terminé normalement (via <code>exit()</code> ou <code>return</code>).
lightblueWEXITSTATUS(status)	Retourne le code de sortie passé à <code>exit()</code> (seulement si <code>WIFEXITED</code> est vrai).

Note : D'autres fonctions sont disponibles pour traiter les cas où l'état du processus fils est modifié par des signaux. Consultez `man wait` pour plus de détails.

5.3 Exemple Complet de Synchronisation

Exemple

Exemple complet avec récupération du statut :

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
```

```
6 int main() {
7     pid_t pid = fork();
8
9     if (pid == 0) {
10         // Processus fils
11         printf("Fils: Je travaille...\n");
12         sleep(2);
13         printf("Fils: Je me termine avec le code 42\n");
14         exit(42);
15     } else if (pid > 0) {
16         // Processus pere
17         printf("Pere: J'attends mon fils (PID: %d)\n", pid);
18
19         int status;
20         pid_t child_pid = wait(&status);
21
22         printf("Pere: Mon fils (PID: %d) s'est termine\n",
23             child_pid);
24
25         if (WIFEXITED(status)) {
26             int exit_code = WEXITSTATUS(status);
27             printf("Pere: Code de sortie du fils: %d\n",
28                 exit_code);
29         }
30     } else {
31         perror("fork a echoue");
32         return 1;
33     }
34
35     return 0;
36 }
```

Sortie attendue :

```
Pere: J'attends mon fils (PID: 12345)
Fils: Je travaille...
Fils: Je me termine avec le code 42
Pere: Mon fils (PID: 12345) s'est termine
Pere: Code de sortie du fils: 42
```

6 Résumé et Bonnes Pratiques

Note Importante

Points clés à retenir :

1. `fork()` crée un processus identique, `exec()` remplace son code
2. Toujours vérifier la valeur de retour de `fork()` (erreur, fils, ou père)
3. **Toujours** utiliser `wait()` ou `waitpid()` pour éviter les zombies

4. Ne jamais utiliser `sleep()` pour la synchronisation
5. L'ordre d'exécution des processus est **non déterministe**
6. Utiliser `waitpid()` quand l'ordre de terminaison est important
7. Récupérer et traiter le statut de sortie avec les macros appropriées

Attention

Erreurs courantes à éviter :

- Oublier de faire `wait()` → Crée des processus zombies
- Ne pas vérifier la valeur de retour de `fork()` → Code instable
- Utiliser `sleep()` pour synchroniser → Non déterministe et peu fiable
- Ne pas gérer le cas `pid < 0` (échec de `fork()`)

7 Tableau Récapitulatif des Fonctions

maincolor !20maincolor !50 Fonction	Rôle	Valeur de retour
maincolor !20 fork()	Créer un processus fils	-1 : erreur 0 : fils >0 : père (PID fils)
maincolor !20 exec*()	Remplacer le code du processus	-1 si erreur Ne retourne pas si succès
maincolor !20 wait()	Attendre un fils quelconque	PID du fils terminé -1 si erreur
maincolor !20 waitpid()	Attendre un fils spécifique	PID du fils terminé -1 si erreur
maincolor !20 exit()	Terminer un processus	Ne retourne pas

Fin du Chapitre 4

Maîtrisez la programmation de processus pour construire des applications robustes !